

LPC 2026

DAMON in the AI Cloud

Monitoring guest workloads on virtualization hosts

PRESENTED BY

Krishna Iyer

October 6, 2026

Why: memory is the blind spot

THE CORE PROBLEM

Mapped vs. Used Memory

Hypervisors know how much guest RAM is mapped — not how much is actually used.

VALUE UNLOCKED

Per-VM Visibility

Enables oversubscription decisions, memory-tiering groundwork, and incident forensics.

OPERATIONAL CONSTRAINT

~Zero Host Cost

Must cost ~nothing on hosts running customer virtual machines.

THE SOLUTION

DAMON Engine

The upstream kernel's data-access monitor — region-based sampling with cost independent of memory size. Maintainer (SJ Park) is at Crusoe.

Why DAMON?

CORE MECHANISM

Adaptive Region Sampling

- **Address Space Regions:** Splits space into regions; checks 1 page per region for accessed bit each interval.
- **Access Reporting:** Each aggregation window reports `nr_accesses` (0–20).
- **Adaptive Scaling:** Regions split/merge adaptively (10–1000 regions at fixed cost).

OPERATING MODES

`paddr` vs. `vaddr`

`paddr` mode

Monitors whole host physical memory via `DAMON_STAT` module.

`vaddr` mode

Monitors one process's virtual address space.

KERNEL AUTO-TUNING

Intervals (6.14+)

- **Dynamic Tuning:** Kernel adjusts sampling between floor & cap to hit target signal ratio.
- **Busy VMs:** Get fine-grained access monitoring.
- **Idle VMs:** Monitoring overhead remains near-free.

DAMON_STAT (paddr) on the host [Aug 2026]

HOST-LEVEL TEST

PASS

Measured bandwidth within 5% of theoretical.

IN-GUEST TEST

FAIL

An 842 GiB load inside a cloud-hypervisor guest was completely invisible.

ROOT CAUSE

Non-LRU Folio Rejection

Guest RAM is 1 GiB hugetlbfs; `damon_get_folio()` rejects non-LRU folios.

FLEET REALITY

Critical Memory Blind Spot

~All guest RAM is 1 GiB hugetlb
→ paddr DAMON was blind to target memory.

Patching paddr: visible, but not discriminating

FLUSH-ASSISTED AGING

aging_flush Series (6 patches)

Flush-assisted aging → load/idle separation restored to **4.4x**, footprint fidelity **82%**.

LIMITATION

1 GiB Granularity Issue

Working vs idle guests looked the same: `clear_young` doesn't flush guest TLBs, so accessed bits re-arm at TLB-refill rate, not access rate.

vaddr profiling

THE FIX (OPTIMIZATION)

Cut CPU Overhead in Half (7% → 3.7%)

- **Initial Problem:** Running updates every 5ms caused ~7% CPU usage.
- **Solution:** Increasing update interval to 100ms dropped CPU to ~3.7% with zero loss in responsiveness.

RESIDUAL OVERHEAD

~3.7% CPU is the Normal Baseline

- **Expected Minimum:** 3.7% is the required baseline cost to sample a huge 2.9 TiB memory space.
- **Zero Idle Cost:** Idle systems consume ~0% CPU.
- **Status:** Validated with data and submitted for maintainer review.

BENCHMARK RESULTS

88x – 184x Separation

Stock 6.17 achieved 88x load/idle separation (run 1), scaling up to 184x at a true 832 GiB hot set (run 2, fixed masim).

KEY TAKEAWAY

How to Measure Kernel Thread Performance Accurately

- **Avoid `ps`:** Standard `ps` tools give inaccurate, unreliable CPU averages for short kernel tasks.
- **Recommended Tool:** Always use `/proc/<pid>/stat` deltas for reliable CPU profiling.

Optimizing kdamond %CPU

CORE MECHANISM

kdamond Does 2 Things

- **Sample memory:** Cheap, and auto-throttles itself when the VM is idle.
- **Tally access stats:** Per-region access stats calculation is expensive.

PREVIOUS INEFFICIENCY

Continuous Tallying

- **Frequent Runs:** Tallying was performed every 5ms / 100ms continuously.
- **Wasted Work:** Data was only being consumed every 30s by the metrics collector.

THE OPTIMIZATION

On-Demand Tally Job

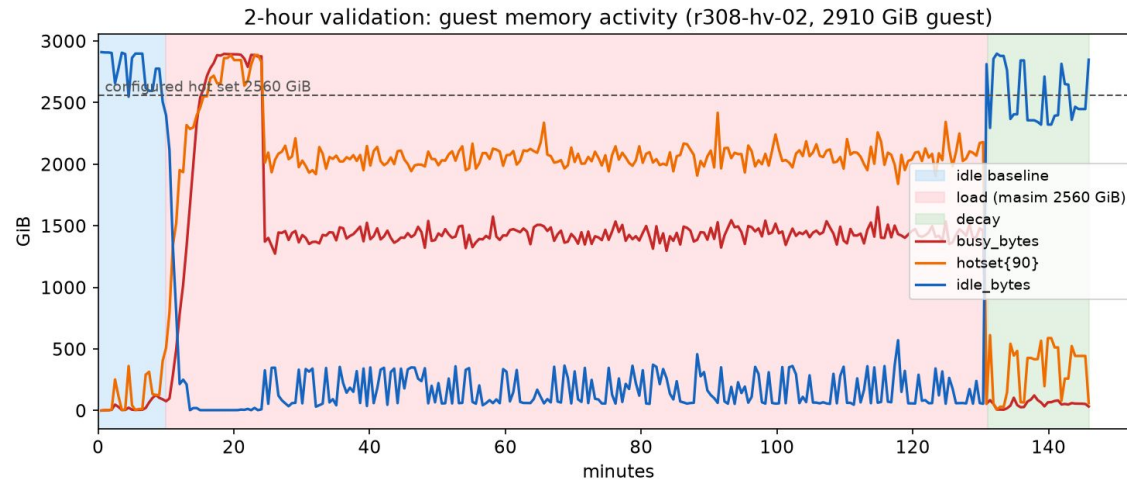
- **Read Triggered:** Runs only when the collector actually reads.
- **Ephemeral Lifecycle:** Installed on demand, run once, and immediately torn down.
- **Idle Efficiency:** Between reads, kdamond does nothing but cheap sampling.

KEY TAKEAWAY

%CPU Reduced by making Tally job on-demand

- Delivers the exact same data with the same fidelity, but at a significantly cheaper CPU cost.

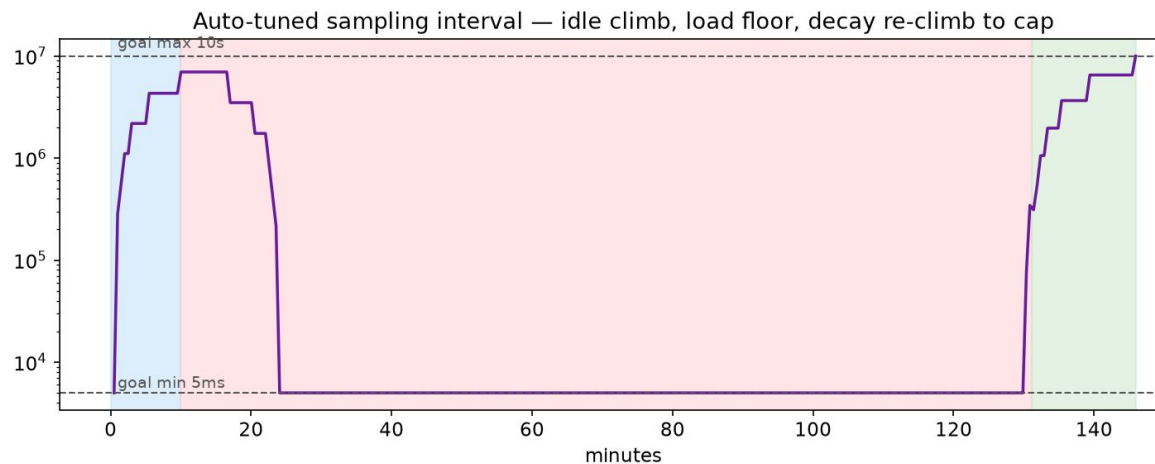
Validation at 2.9 TiB (B200 node)



MEMORY ACTIVITY VALIDATION

2.91 TiB Guest Load

- **2,910 GiB Guest Memory** tested on r308-hv-02 node.
- **2,560 GiB** configured hot set target.
- **Distinct Phases:** Clear separation across idle baseline, masim active load, and decay.



SAMPLING INTERVAL RESPONSE

Auto-Tuning Performance

- **5 ms Floor:** Reached rapidly during high memory load phase.
- **10 s Cap:** Sustained during idle climb and post-load decay.
- **Zero Manual Writes:** Collector dynamically adjusted sampling rate end-to-end.

Reading the metrics (the simple version)

Metric	What it means
busy_bytes	How much of the VM's memory is being actively used (constant use counts fully, occasional use partially)
idle_bytes	How much wasn't touched at all recently
hotset_bytes{90}	The smallest chunk of memory doing 90% of the work

'Recently' = the auto-tuned window (0.1–200s). busy is a lower bound for very large hot sets; hot-set coverage stays accurate.

Discussion Points

01 / CPU SAMPLING

`min_sample_us` & CPU Budget

`min_sample_us` is the de-facto CPU ceiling for sampling overhead.

KEY QUESTION

Is a CPU-aware intervals goal (tune-to-CPU-budget) worth building?

02 / MULTI-TARGET ATTRIBUTION

`tried_regions` Scaling

A per-target results directory would let one `kdamond` serve many VMs.

KEY QUESTION

Is there sufficient interest to build multi-VM target attribution?

03 / HUGE PAGE MONITORING

`hugetlb` & TLB Staleness

`paddr` monitoring patches + flush-assisted aging (`clear_flush_young`) are RFC-ready modulo re-validated fidelity numbers.

KEY QUESTION

How should the TLB-staleness trade-off be surfaced upstream?